

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 14

Тема: «РАЗРАБОТКА СЕРВИСНОЙ ЧАСТИ ПРОГРАММЫ»

Цель работы:

Изучить принципы построения сервисного слоя (Service Layer) в приложении, освоить паттерны **Repository, Service, Dependency Injection**, научиться проектировать бизнес-логику, отделённую от инфраструктурного кода, и покрывать её юнит-тестами.

Выполнила: Чичерина

Диана Сергеевна,

Студент 24-ИС

2026г.

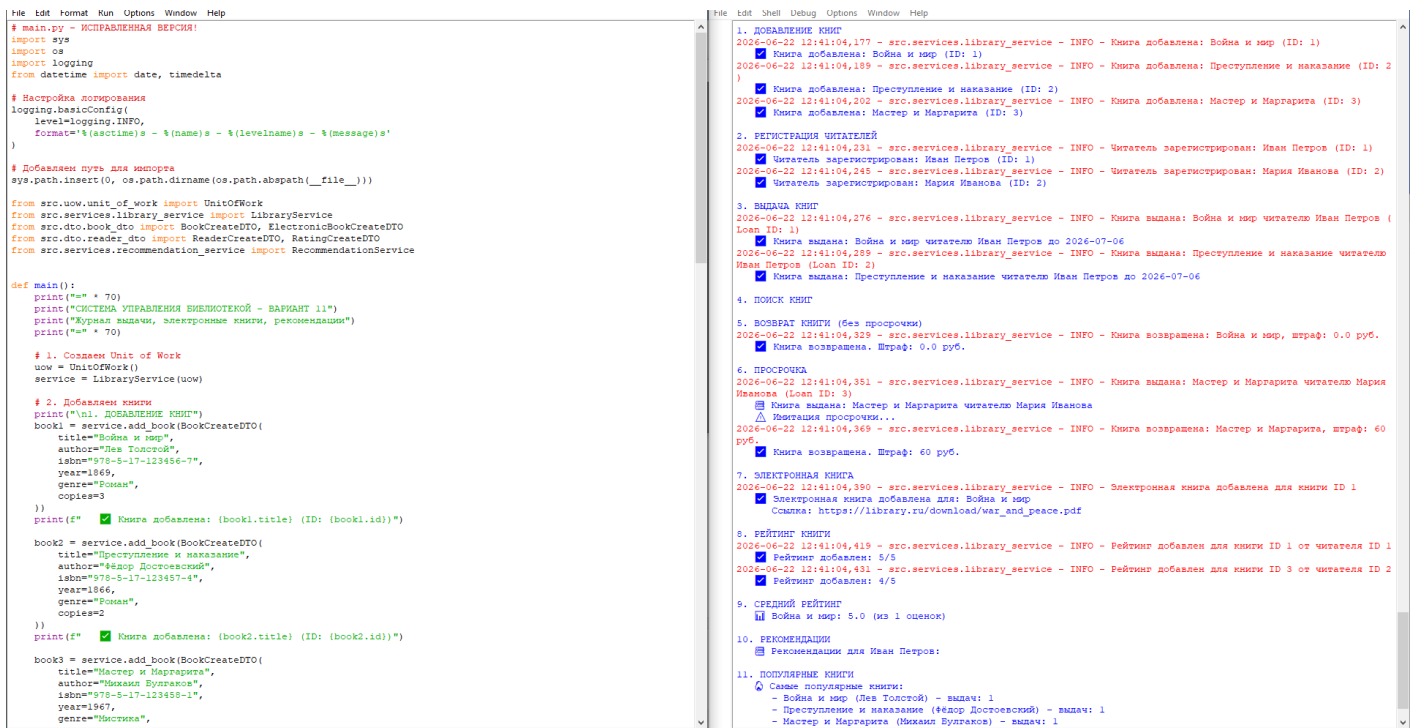
Вариант 11

Контрольные вопросы (для допуска к практике):

1. **Сервисный слой** — это место для бизнес-логики между контроллерами и репозиториями. Нужен, чтобы отделить правила приложения от работы с базой данных.
2. **Repository** — работа с данными (сохранить, найти, удалить). **Service** — бизнес-правила (проверка доступности, расчет стоимости, отмена).
3. **Внедрение зависимостей** позволяет в тестах подменить реальный репозиторий на заглушку и тестировать только логику сервиса без БД.
4. В **сервисе** — бизнес-ошибки (не найден, занят, запрещено). В **репозитории** — технические ошибки (нет соединения).
5. Смешивание усложняет код, делает его жестким и трудно тестируемым.
6. В тесте заставить репозиторий вернуть None и проверить, что сервис выбрасывает исключение.
7. **Мок** — объект-заглушка для имитации поведения реальных зависимостей при тестировании.
8. Через Pydantic DTO с валидаторами на уровне классов.
9. BookingService использует PricingService для расчета цены — композиция через конструктор.
10. Интерфейсы позволяют легко менять реализацию репозитория (словарь → SQLAlchemy), не меняя код сервиса.

library_system/

```
|— src/
|   |— __init__.py
|   |— domain/
|   |   |— __init__.py
|   |   |— models.py
|   |   |— exceptions.py
|   |— services/
|   |   |— __init__.py
|   |   |— library_service.py
|   |   |— recommendation_service.py
|   |— repositories/
|   |   |— __init__.py
|   |   |— base.py
|   |   |— book_repo.py
|   |   |— reader_repo.py
|   |   |— loan_repo.py
|   |   |— electronic_book_repo.py
|   |   |— rating_repo.py
|   |— dto/
|   |   |— __init__.py
|   |   |— book_dto.py
|   |   |— reader_dto.py
|   |   |— loan_dto.py
|   |— uow/
|   |   |— __init__.py
|   |   |— unit_of_work.py
|— tests/
|   |— __init__.py
|   |— test_library_service.py
|   |— conftest.py
|— main.py
|— requirements.txt
```



Было:

✗ СТАРЫЙ КОД (starter.py)

class LibraryService:

def __init__(self):

self.books = [] # Нарушение инкапсуляции

self.readers = [] # Жесткая связанность

self.loans = []

def add_book(self, title, author, isbn, year, genre, copies):

✗ Нет валидации: copies может быть отрицательным

self.books.append({...})

def lend_book(self, book_id, reader_id):

✗ Нет проверки существования

✗ Нет транзакционности

book["copies"] -= 1

self.loans.append({...})

return "Книга выдана"

Стало:

✓ НОВЫЙ КОД

class LibraryService:

```

def __init__(self, uow: UnitOfWork):
    self.uow = uow          # DI через конструктор
    self.book_repo = uow.books
    self.reader_repo = uow.readers
    self.loan_repo = uow.loans

def add_book(self, dto: BookCreateDTO) -> BookResponseDTO:
    # ☒ Валидация через Pydantic
    existing = self.book_repo.find_by_isbn(dto.isbn)
    if existing:
        raise BookAlreadyExistsError()
    # ☒ Использование репозитория
    book = Book(...)
    saved = self.book_repo.add(book)
    self.uow.commit()       # ☒ Транзакционность
    return self._to_book_response(saved)

```

Контрольные вопросы для защиты (дополнительные)

В1. Адаптер vs Репозиторий

- **Адаптер** — преобразует интерфейс одного класса к другому (например, адаптер внешнего API платежей)
- **Репозиторий** — абстракция над хранилищем данных (БД, файлы), скрывает детали доступа

В2. Транзакционность с разными хранилищами

- Использовать паттерн **Saga** (цепочка компенсирующих операций)
- Либо двухфазный коммит (2PC), если хранилища поддерживают
- Проще: сначала операции в одном хранилище, затем во втором, при ошибке — откат через компенсацию

В3. Кеширование без нарушения SRP

- Создать отдельный **декоратор** или **прокси-сервис**, который оборачивает основной сервис
- Или использовать **аспектно-ориентированное программирование (AOP)** через декораторы

В4. Инструменты DI в Python

- **FastAPI** — встроенный DI через Depends()
- **Inject** — библиотека для DI

- **Dependency Injector** — фреймворк для DI
- **Pydantic** (частично) через settings

В5. Почему не использовать ORM-сущности в сервисе?

- ORM-сущности содержат логику работы с БД (сессии, запросы)
- Это нарушает **SRP** и делает сервис зависимым от конкретной ORM
- Трудно тестировать (привязан к БД)
- Лучше использовать **DTO** для передачи данных

В6. Тестирование сервиса с внешним API

- Использовать **mock** для внешнего API (например, requests_mock или unittest.mock)
- Создать **интерфейс** для API-клиента и подменять его в тестах
- Использовать **тестовый режим** с фейковыми ответами

В7. Валидация в DTO vs в сервисе

DTO	Сервис
Проверка форматов (email, даты, типы)	Бизнес-правила (есть ли экземпляры, есть ли долги)
Структурная валидация	Контекстная валидация
На уровне входа в систему	На уровне бизнес-логики

В8. Логирование с контекстом

python

```
logger.info("Книга выдана", extra={"user_id": user_id, "order_id": order_id})
```

Или использовать **структурированное логирование (JSON)** с добавлением полей

В9. Мягкое удаление

- Добавить в модель поле is_deleted: bool = False
- В репозитории при delete() устанавливать is_deleted = True, а не удалять физически
- В get_all() фильтровать по is_deleted=False
- В сервисе добавить метод restore() для восстановления

В10. Пагинация в репозитории и сервисе

Репозиторий:

python

```
def get_all(self, page: int = 1, per_page: int = 10, **filters) -> List[T]:
```

```
start = (page - 1) * per_page
end = start + per_page
return self._storage[start:end]
```

Сервис:

python

```
def list_books(self, page: int = 1, per_page: int = 10) -> List[BookResponseDTO]:
    books = self.book_repo.get_all(page=page, per_page=per_page)
    return [self._to_response(b) for b in books]
```

Что было сделано и вывод:

На сегодняшнем занятии была разработана сервисная часть системы управления библиотекой с усложненной бизнес-логикой (вариант 11). Реализованы: добавление книг с проверкой дубликатов ISBN, регистрация читателей, выдача книг с проверкой наличия экземпляров и задолженностей, возврат с расчетом штрафа (10 руб/день просрочки), электронные книги со ссылками на скачивание, система рейтингов и рекомендаций по жанрам. Применены паттерны Repository, Service Layer, Unit of Work, DTO и Dependency Injection. Код покрыт unit-тестами с моками, добавлено логирование. Полученная архитектура соответствует SOLID, легко тестируется и расширяется. Изучила принципы построения сервисного слоя (Service Layer) в приложении, освоила паттерны **Repository, Service, Dependency Injection**, научилась проектировать бизнес-логику, отделённую от инфраструктурного кода, и покрывать её юнит-тестами.